CHAINSECURITY

PUBLIC

Code Assessment
of the Grove Periphery Smart Contracts

September 07, 2026

Produced for



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	5
4	Trust Model	7
5	System Considerations	8
6	Terminology	9
7	Findings	10
8	Limitations and use of report	12

1 Executive Summary

Dear all,

Thank you for trusting us to help Grove with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Grove Periphery according to **Scope** to support you in forming an opinion on their security risks.

Grove implements a peripheral contract that converts USDC held by governance into USDS at a rate of one to one. The conversion runs through the Sky DAI LitePSM and the DAI to USDS exchanger, is available in one direction only, and returns the output to the same address the USDC was pulled from.

The most critical subjects covered in our audit are functional correctness, security of assets and access control. Both hops of the conversion are checked: the DAI received must equal the amount owed at parity, and the holder's balance must rise by exactly that amount, so a sell fee on the PSM or a leg that does not deliver reverts the call rather than settling short. The contract custodies nothing between calls and pays only to the holder, so the exposure of the three roles is bounded by the allowance the holder grants. Security regarding all the aforementioned subjects is high.

The general subjects covered are documentation, unit testing and trustworthiness. The repository documents the contract, its roles and its deployment in detail. Security regarding all the aforementioned subjects is good.

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commits which are referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Grove Periphery repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	20 August 2026	4259ca4cfb3a92829a2b5253f99c1a20a4ef84b9	v1.0.0.beta-1
2	7 September 2026	45121058b08092ff1073d0885026ed3d188f102c	Fix review

For the Solidity smart contracts, the compiler version `0.8.34` was chosen.

The scope consists of the converter, the library that deploys it and the scripts that deploy and verify a deployment:

```
deploy/
└── InstantUsdcUsdsConverterDeploy.sol
script/
├── DeployInstantUsdcUsdsConverter.s.sol
└── VerifyInstantUsdcUsdsConverter.s.sol
src/
└── InstantUsdcUsdsConverter.sol
```

2.1.1 Excluded from scope

All other files in the repository, including the tests, are out of scope. The DAI LitePSM and the DAI to USDS exchanger that the converter routes through are out of scope and assumed to behave as documented. Additionally, all dependencies are out of scope.

3 System Overview

This system overview describes the most recently received version (**Version 2**) of the contracts as defined in the Assessment Overview.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Grove offers a peripheral contract that converts USDC held by governance into USDS at a rate of one to one. The conversion is not available in the opposite direction.

3.1 InstantUsdcUsdsConverter

The converter routes a swap through two Sky contracts: the DAI LitePSM, which sells USDC for DAI, and the DAI to USDS exchanger, which turns that DAI into USDS. Both are supplied at construction, as is the `holder`, the Grove governance proxy that funds every swap and receives every output.

The remaining addresses are derived rather than passed in. The converter reads `gem`, `dai` and `to18ConversionFactor` from the LitePSM and `dai` and `usds` from the exchanger, and requires the two to agree on DAI. `to18ConversionFactor` is stored as `conversionFactor` and scales a USDC amount to the USDS amount an exact conversion owes. Every one of these values is immutable, so a deployment is bound to one PSM, one exchanger, one token triple and one holder for its lifetime.

3.1.1 Swapping

`swap(usdcAmount)` is gated on `SWAPPER_ROLE` and performs the whole conversion in one call:

1. It rejects a zero amount and records the holder's USDS balance.
2. It pulls `usdcAmount` USDC from the holder, which must have approved the converter beforehand, and approves that amount to the LitePSM.
3. It picks a route by reading `bud` on the LitePSM: `sellGemNoFee` when the converter sits on the PSM allowlist, and the permissionless `sellGem` otherwise. Both send the DAI to the converter.
4. It resets the PSM allowance to zero, covering the case where the PSM did not pull the full amount.
5. It requires the DAI received to equal `usdcAmount * conversionFactor` exactly. A sell fee on the PSM makes the call revert rather than settle short, which is why the fee free route matters once Sky governance charges one.
6. It approves the DAI to the exchanger, calls `daiToUsds` with the holder as the recipient, and resets that allowance to zero.
7. It requires the holder's USDS balance to have risen by exactly the DAI amount, so a leg that reports success without delivering is caught.

The PSM allowlist entry belongs to the converter rather than to its callers, so a single `kiss` from Sky covers every account holding `SWAPPER_ROLE`. Which accounts those are is Grove's decision alone, making the role a whitelist maintained on top of Sky's single grant.

The converter is therefore a pass through: the USDC and the intermediate DAI both leave within the same call, and no balance or allowance is meant to survive it. The amount of USDS delivered is returned and emitted, together with the route that was taken.

3.1.2 Roles

Access control is OpenZeppelin `AccessControl` with three roles, all granted in the constructor.

`DEFAULT_ADMIN_ROLE` manages the roles and performs the rescue. `SWAPPER_ROLE` is the only role that can call `swap`. `FREEZER_ROLE` can call `removeSwapper`, which revokes `SWAPPER_ROLE` from an account as an emergency stop; it cannot grant the role back, so reinstating a swapper is an admin action.

3.1.3 Rescue

A single admin function recovers tokens that reached the converter by mistake: `rescueERC20` transfers the contract's entire balance of a given token to the holder. There is no path for recovering ETH, and the contract is not payable. An ERC721 is likewise unrecoverable.

3.2 Deployment

Deployment goes through the `InstantUsdcUsdsConverterDeploy` library, which supplies the converter's constructor arguments. The mainnet script is gated on a chain id of one and draws those arguments from constants of the `grove-address-registry` dependency: `GROVE_PROXY` as both admin and holder, `ALM_RELAYER` as swapper and `ALM_FREEZER` as freezer, alongside the LitePSM and the exchanger.

All three roles are granted in the constructor directly to their final holders, so the deploying account never holds a role and no handover follows. A separate read-only script checks the wiring and the role holders of a live converter after deployment.

3.3 Changelog

In **Version 2**, the recovery function `rescueERC721` for rescuing ERC721 tokens was removed.

4 Trust Model

This trust model covers the `InstantUsdcUsdsConverter`. The contract custodies nothing by design: each swap pulls from the holder and pays back to the holder within the same call, so compromising any role exposes the allowance the holder has outstanding rather than a balance.

4.1 Roles

Admin (`DEFAULT_ADMIN_ROLE`): Fully trusted. It manages all three roles and performs the rescue. `rescueERC20` hardcodes the holder as the destination, so the admin cannot direct funds elsewhere; the worst it can do is grant `SWAPPER_ROLE` and force conversions. On the mainnet wiring the admin and the holder are the same Grove governance proxy.

Swapper (`SWAPPER_ROLE`): Partially trusted. It decides when and in what size the holder's USDC is converted. The contract applies no limit of its own, so the only bound is the allowance the holder has granted the converter. Because the conversion is enforced at one to one and the output is delivered to the holder, a compromised swapper cannot extract value: it can convert USDC the holder intended to keep, and it can exhaust that allowance. Because the PSM allowlist entry is held by the converter, a swapper can use the fee free sell route for as long as it holds the role, so Grove controls that access through `SWAPPER_ROLE` without a further grant from Sky.

Freezer (`FREEZER_ROLE`): Partially trusted. It can revoke `SWAPPER_ROLE` from any account and thereby halt conversions. It cannot grant the role back, cannot swap and cannot rescue, so this is a denial of service capability only.

4.2 External Components

DAI LitePSM: Trusted to sell USDC for DAI at the factor it reports. Sky governance controls both its sell fee and its allowlist, and neither is taken on trust: a nonzero fee makes the one to one check fail, and removing the converter from the allowlist silently moves the swap to the permissionless route, which then fails in the same way once a fee is charged. A PSM that accepted the USDC without paying out the DAI fails that check as well, and because the conversion is a single call the revert returns the USDC. Selling can also be halted outright, and a dedicated mom contract lets Sky do so without waiting for the governance delay, which stops conversions on both routes at no notice.

DAI to USDS exchanger: Trusted to deliver USDS to the holder against the DAI it is given. Non delivery is caught by comparing the holder's USDS balance before and after rather than assumed away, so this leg also fails closed.

USDC, DAI and USDS: Assumed to behave as standard ERC20 tokens. USDC is administered by its issuer and can freeze accounts; a freeze on the converter or on the holder blocks conversions, but it does not put funds at risk, as the converter holds no balance between calls.

Holder: Assumed to be the Grove governance proxy, and the only address the contract ever pays. It is trusted to size its allowance to the converter deliberately, since that allowance is precisely what a swapper is able to spend.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 Swap Size Is Bounded by the LitePSM's DAI Balance

System Consideration | Version 1

The LitePSM pays out from DAI it holds in advance, which a permissionless `fill` tops back up to a configured buffer. `swap` never calls `fill`, so a swap larger than the balance the PSM holds at that moment reverts inside the PSM rather than in the converter.

Sizing a swap to the balance available, or calling `fill` beforehand, is therefore left to the caller.

SC2 Deployment Validation

System Consideration | Version 1

The deployer is untrusted and the deployment and verification scripts do not establish that the intended contract was deployed correctly. Besides bytecode verification and the verification scripts, governance should ensure that no unexpected role assignments were performed.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

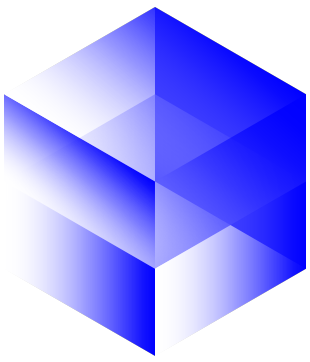
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	**Impact**		
	High	Medium	Low
High	**Critical**	**High**	**Medium**
Medium	**High**	**Medium**	**Low**
Low	**Medium**	**Low**	**Low**

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

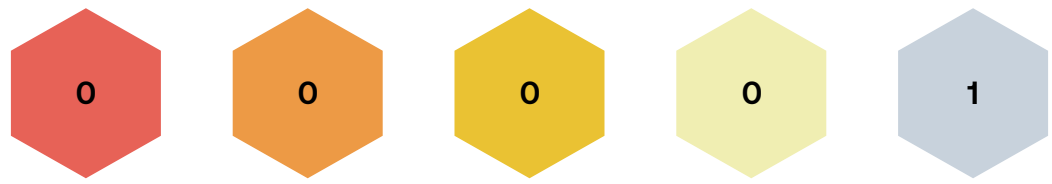
7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



No Open Issues

Total number of findings



Critical | High | Medium | Low | Informational

7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Non-informational findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

Info	#001	Unnecessary ERC721 Rescue	Code Corrected ✓

7.2 Descriptions

#001 Unnecessary ERC721 Rescue

Informational	Version 1	Code Corrected

`rescueERC721` recovers an ERC721 stuck in the converter by transferring the token id to the holder. Nothing in the swap path involves an ERC721; the converter interacts only with USDC, DAI, USDS, the LitePSM and the DAI to USDS exchanger. The contract does not implement `onERC721Received`, so a `safeTransferFrom` to it reverts, leaving a plain `transferFrom` or a mint as the only way one can arrive.

The function is also not limited to ERC721. `transferFrom(address,address,uint256)` is the same selector for both standards, so passing an ERC20 transfers that amount of tokens and emits `ERC721Rescued` with the amount reported as a token id.

Code Corrected

The function was removed.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.